Public

Unified Telemetry for Next-Generation Rack-Scale Infrastructure

Antonis Michaloliakos Production Systems Engineer, AI Systems, Meta
Nischay Tata Fellow, Systems Design Engineer, AMD

Public

Agenda

01 **The observability cliff**
What we lose when the PCIe switch disappears

02 **The observer effect**
How telemetry polling generates the congestion it measures

03 **Unified telemetry architecture**
In-band (amd-smi) · OOB (Redfish / PLDM) · safeguards

04 **Three operational pillars**
Deep debugging · job scheduling · fleet health

05 Helios proving ground
Backpressure localisation on MI450, from active bring-up campaigns

06 **Path to an open standard**
A vendor-neutral boundary-telemetry schema for OCP

Public

Building on OCP EMEA 2026

We named the problem at EMEA. Since then we have validated the fix on real racks. Today we propose it as a standard.

OCP EMEA 2026 — SINCE THEN — TODAY · OCP GLOBAL 2026 — NEXT

Legacy telemetry challenges

- Defined the observability cliff
- Discovered the observer effect

New-age telemetry

- Implementation in the field
- Validated across multiple rack platforms and campaigns

Unified telemetry architecture

- Field-proven architecture
- Three operational pillars

Open standard

- Vendor-neutral schema
- OCP working group

Public

Four findings that drive this work

What the legacy telemetry architecture taught us, and why it matters more at rack scale.

01

Polling was the congestion

High-rate telemetry polling caused most of the fabric congestion it was measuring, in key scenarios.

02

One host stalls 999

One congested host in a 1,000-GPU job stalls all 999 others. **Observability = utilization.**

03

The probe point is gone

Helios removes the external PCIe switch: the diagnostic tool that solved the last problem no longer exists.

04

The bottleneck is unattributable

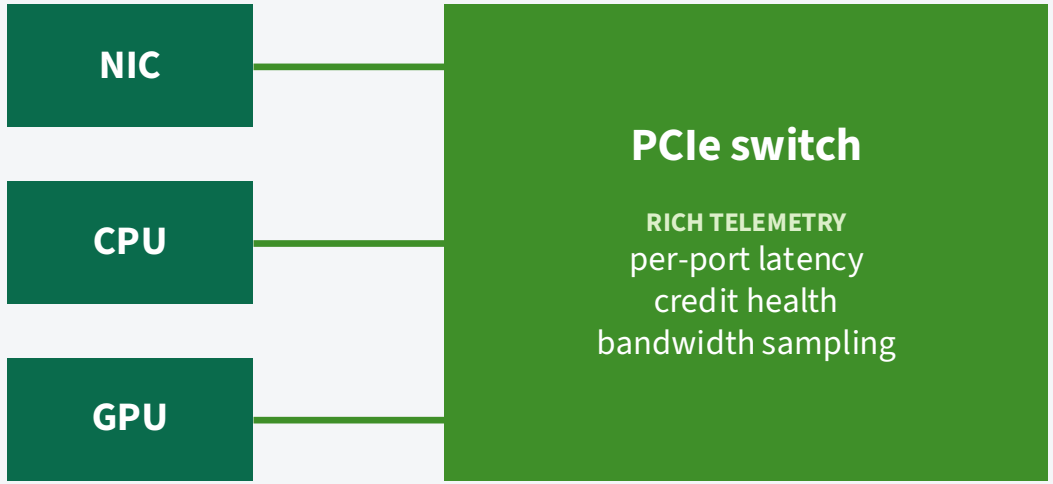
Without interface telemetry, engineers cannot tell PCIe, CPU memory, GPU, Data Fabric or scale-up link apart.

Public

Removing the PCIe switch removes our richest probe point

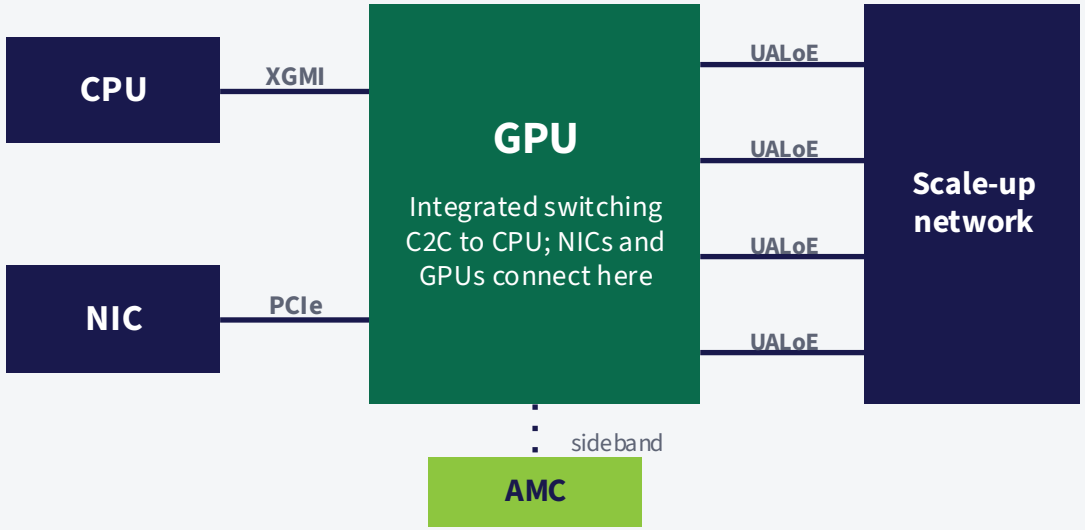
Today the external PCIe switch doubles as our best instrument. Helios integrates it away.

CURRENT GENERATION



✓ **One external switch = one rich probe point on every link**

NEXT GENERATION · HELIOS



⚠ **Observability gap:** no external switch = no probe point

More integration = faster silicon, but less observable silicon.

Public

The observer effect

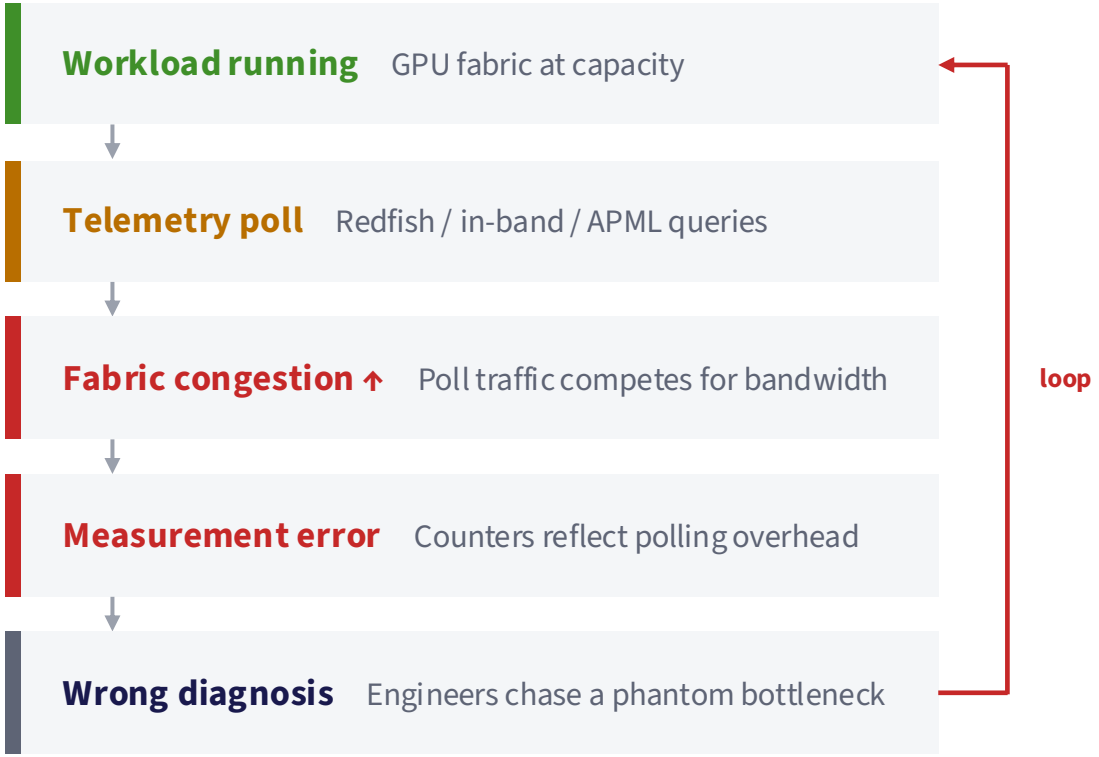
At high polling rates, telemetry generated the very back pressure it was trying to measure.

WHAT WE MEASURED

- Partial cache-line writes from telemetry reads saturated the CPU memory subsystem
- High-frequency monitoring generated backpressure indistinguishable from workload congestion

WHAT THE DESIGN MUST STILL DELIVER

- Node health and utilization, to drive job scheduling and maintenance
- Reliable OOB telemetry, even when in-band congestion gates both the workloads and the telemetry



Monitoring became the congestion it was measuring.

Public

Unified telemetry architecture: three layers, all required

In-band gives resolution, out-of-band gives persistence, and the safeguards keep the first from recreating the observer effect.

In-band telemetry
amd-smi

- **GPU:** clocks, power, temperature, utilization
- **CPU:** power, thermal, frequency, memory
- **NIC:** network & port statistics, congestion
- **RAS:** ECC and RAS event counters
- **Node:** board sensors, VR thermal, IBC temp
- **Scale-up:** UALoE metrics
- **HSIO:** SerDes link state

Out-of-band telemetry
Redfish / PLDM

- **BMC:** tray thermals, CPU sensors, chassis health
- **AMC:** GPU health, GPU FW inventory, EventLog
- **NIC:** port statistics, health state
- **Switch:** fabric health, port state
- **Rack manager:** rack thermal, leak detection
- **Scale-up:** threshold crossing, bulk metrics

Architectural safeguards
both paths

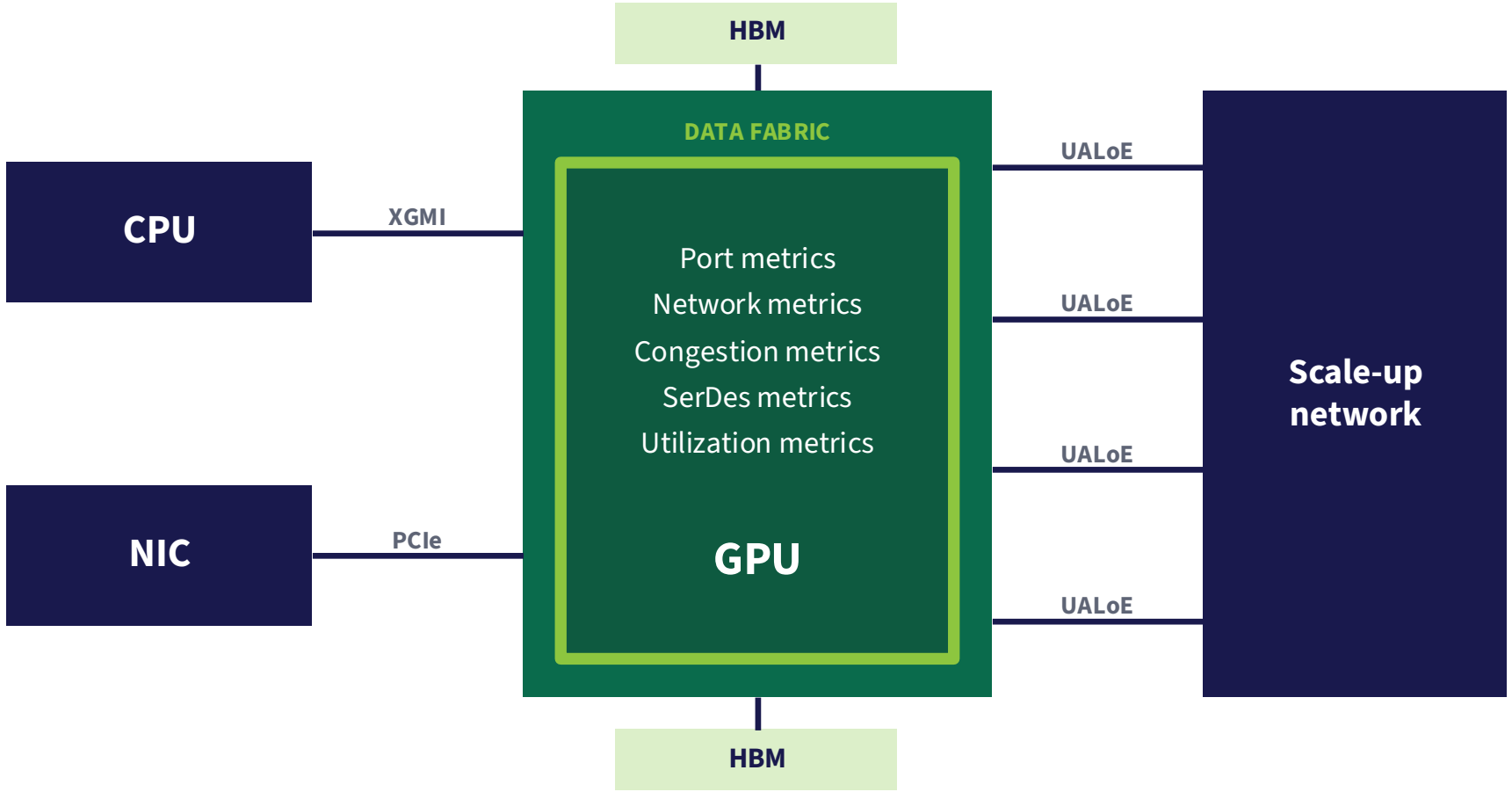
- **Priority:** telemetry at lowest Data Fabric priority
- **Adaptive polling:** back off in-band under heavy load, lean on OOB
- **Affinity:** telemetry threads off compute cores
- **Verbosity tiers:** production vs. debug, enforced
- **OOB first:** out-of-band as the priority path
- **Qual gate:** telemetry-impact benchmarking

All three layers are required: they are complementary, not alternatives.

Public

Helios: where traffic and load telemetry lives

Every link (XGMI, PCIe, UALoE, HBM) terminates on the GPU's Data Fabric, which makes it the new probe point.



Public

Guardrails: a tiered verbosity

The verbosity tier is selected per operator role and enforced at the collection plane, never ad hoc.

Production mode

Near-zero overhead · always-on · no operator action required

- Node health for scheduling, plus workload signs-of-life at runtime
- Polling at ~1 s (relaxable to 10 s)
- Port & network metrics (exposed today); coarse backpressure inferred from credit / buffer counters
- Near-zero by design: a prioritized minimal set (nothing is truly zero-cost)

EXAMPLES

GPU: utilization, power, temp, throttle state

Fabric: backpressure flag (buffer-full counter), credit high-water

OOB: GPU power-good (reached S0?), AMC health rollup, threshold events

Debug mode

On-demand · operator-gated · off by default to avoid the observer effect

- Comprehensive profiling, deep debug and boot-failure root cause
- Polling down to 500 ms; a concentrated metric subset down to low-teens ms
- Full metric set plus high-frequency transients; opt-in filesystem copy / DiagLog bundle
- Scoped to debug windows: off in production to avoid the observer effect

EXAMPLES

GPU: per-engine activity, per-HBM ECC, throttle-reason registers

Fabric: DF credits per direction, buffer depth, DF / PCIe congestion & credits

RAS / boot: APML decode, boot codes 0xBA / 0xB0 / 0xBF / 0xA1, SerDes lstate

Public

Combined in-band + out-of-band collection

Two independent collection paths, each covering the other's blind spot.

OUT-OF-BAND

Operator / automation	BMC	AMC	GPUs	Archive
Local host DC infra	SSH + Redfish CPU telemetry AINIC telemetry	GPU mgmt plane GPU telemetry	GPU subsystems RAS metrics	Timestamped bundles

IN-BAND

Operator / automation	Host OS	NIC	Scale-out	Compute
Local host DC infra	amd-smi fine-grain GPU / CPU telemetry	nic-ctl network / HSIO metrics	afm-ctl network / congestion metrics	Utilization metrics

✓ **Recommended: combined in-band + OOB**

OOB anchors persistent state; in-band captures high-frequency transients at low overhead.

Public

Pillar 1 · Deep debugging: node boot failures

Boot-loop root-cause flow, captured live during Helios bring-up.



SerDes 'Not Yet Up'

- Multiple units: initial FW drops
- Persistent state values across all runs
- Same GPUs affected across campaigns

GPU failures

- PG registers: GPUs not reaching S0
- Detected via OOB telemetry

Firmware load error

- ERR_FIRMWARE_LOAD on affected GPUs
- Boot error code decoded from APML registers
- Consistent pattern across campaigns

XGMI link-training failure

- ERR_XGMI_LINK_TRAINING observed
- OUTBNDMSG decoded per GPU
- Correlated with SerDes values

Public

Pillar 2 · Intelligent job scheduling

Boot state and runtime events decide, per node, whether the scheduler may dispatch work.

GPU STATE → SCHEDULING DECISION

GPU STATE	SCHEDULING DECISION
Boot complete, telemetry healthy Add to scheduling pool	✓ Schedulable
Boot progress codes Hold; retry after timeout	⏳ Pending
Runtime error codes Drain workload, raise watchdog event	⚠ Drain
Boot-time error codes Remove from pool, flag for RMA	✗ Remove
No telemetry response Check power-good register, FRU identity	? Investigate

Watchdog events

Signals → severity → action

Passive telemetry + active probes catch “gray” degradation before hard failure

⚠ **Warning: soft-avoid**
Deprioritize; host stays schedulable

⛔ **Critical: hard-disable**
Drain job, checkpoint & migrate

✓ **Recovery: rejoin**
N passes + preflight before re-enable (anti-flap)

Bounded blast radius: capacity floor + circuit breaker

Scheduler polls the EventLog to gate dispatch decisions

Public

Pillar 3 · Fleet health monitoring

Every run auto-generates a per-node, per-interface health matrix: single-node faults stand out, and so do fleet-wide patterns.

	PCIe	XGMI	UALoE	CPU	GPU	NIC
Node-01	PASS	PASS	PASS	PASS	PASS	PASS
Node-02	PASS	PASS	WARN	PASS	PASS	PASS
Node-03	PASS	PASS	PASS	PASS	PASS	PASS
Node-04	PASS	PASS	PASS	PASS	PASS	PASS
Node-05	PASS	PASS	PASS	PASS	PASS	PASS
Node-06	FAIL	PASS	PASS	PASS	PASS	PASS
Node-07	PASS	PASS	PASS	PASS	PASS	PASS
Node-08	PASS	PASS	FAIL	PASS	PASS	PASS
Node-09	PASS	PASS	PASS	PASS	PASS	PASS
Node-10	PASS	PASS	PASS	PASS	WARN	PASS
Node-11	PASS	PASS	PASS	PASS	PASS	PASS
Node-12	PASS	PASS	PASS	PASS	PASS	PASS
Node-13	PASS	PASS	PASS	PASS	PASS	PASS

1,790
total bundles

1,729
HSIO bundle runs

52
compute test types

6
HSIO test categories

Multiple nodes monitored; reports auto-generated per run

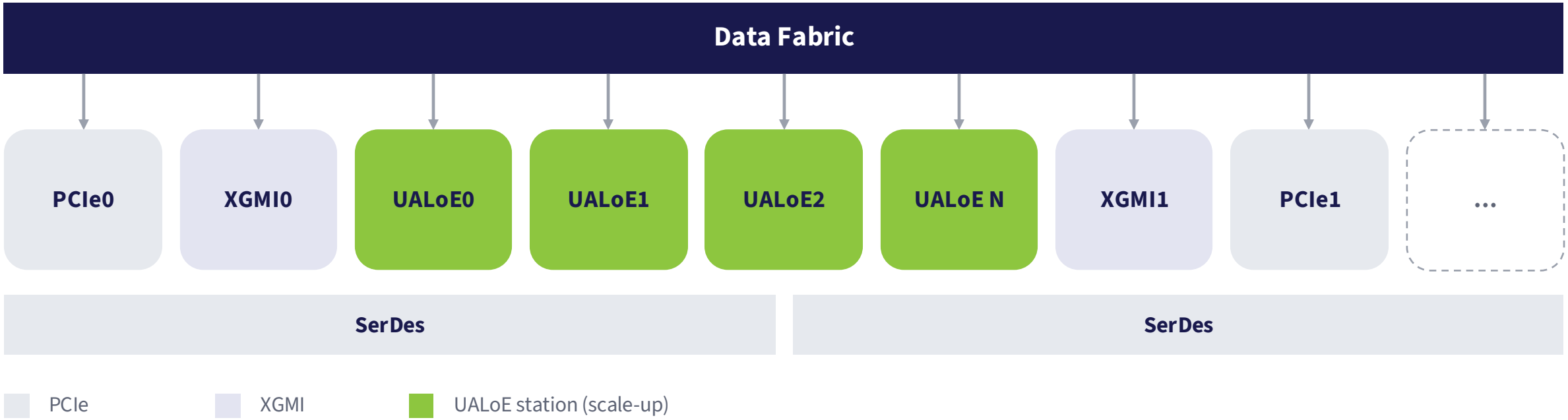
⚠ Systemic issue found

Compute workload timeout
→ platform bug, not node-specific

Public

Where the counters live: the MI450 Helios platform

One Data Fabric, and every scale-up port hangs off it. Multiple UALoE stations per GPU.

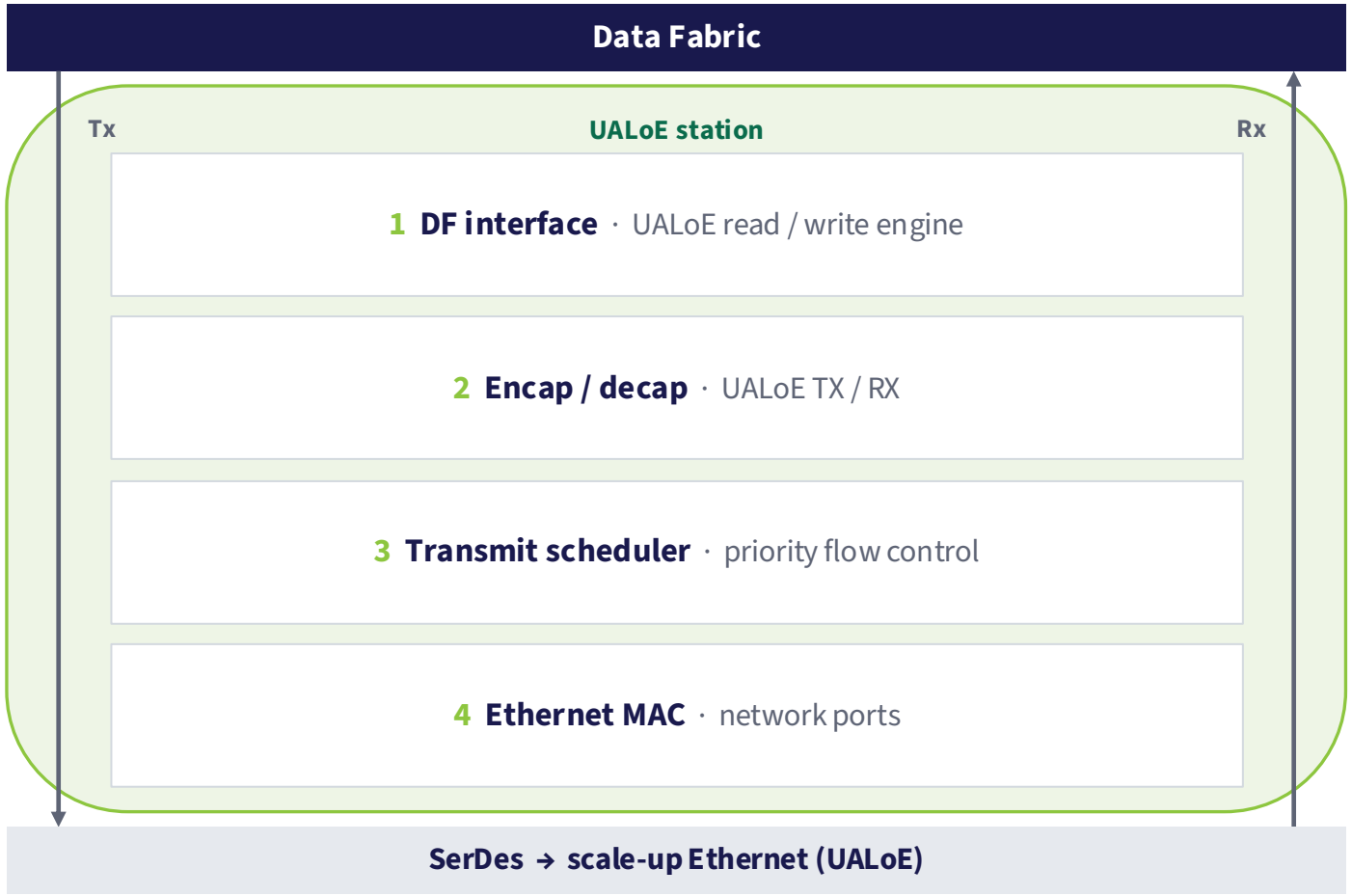


Tens of Data Fabric counters per station on every GPU, which is exactly why the always-on set has to be small.

Public

Inside one UALoE station

Four stages between the Data Fabric and the wire. Each one can stall, and each one stalls differently.



Four stages, four ways to stall

1 DF interface
Moves data between the Data Fabric and the station

2 Encap / decap
Packs and unpacks UALoE packets

3 Transmit scheduler
Arbitrates egress; applies priority flow control

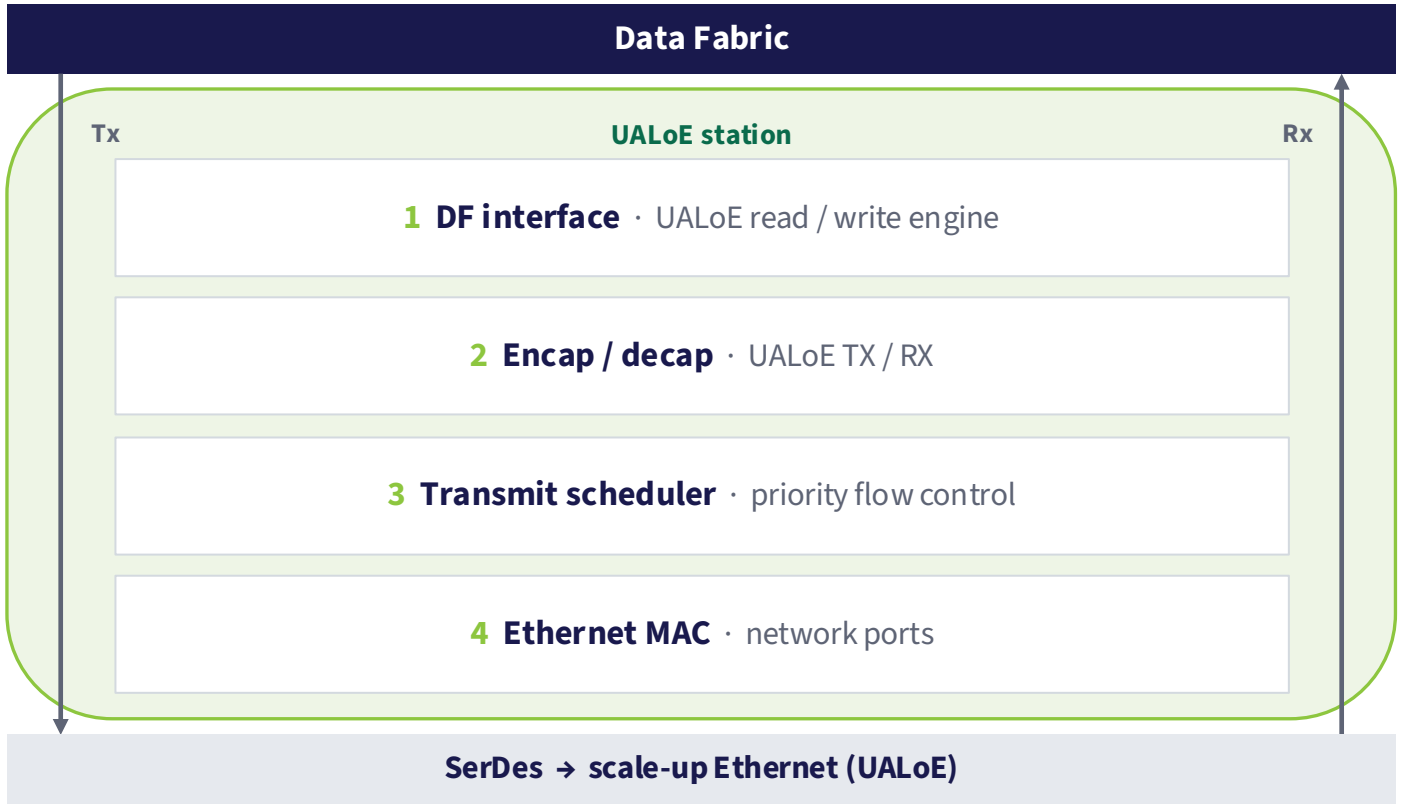
4 Ethernet MAC
Network ports onto the SerDes

Tx flows down the stack; Rx flows up.

Public

A full counter set at every stage, available today

Hundreds of counters spanning all four stages, read on Helios today. This is the reference the standard should require everywhere.



Per-stage counter set

TX

- ingress buffer full / empty
- packetization buffer full / empty
- request / response pool full
- credit consumed / released
- per-class error count

RX

- cycles stalled (+ count)
- payload / excess credits
- tag allocate / free

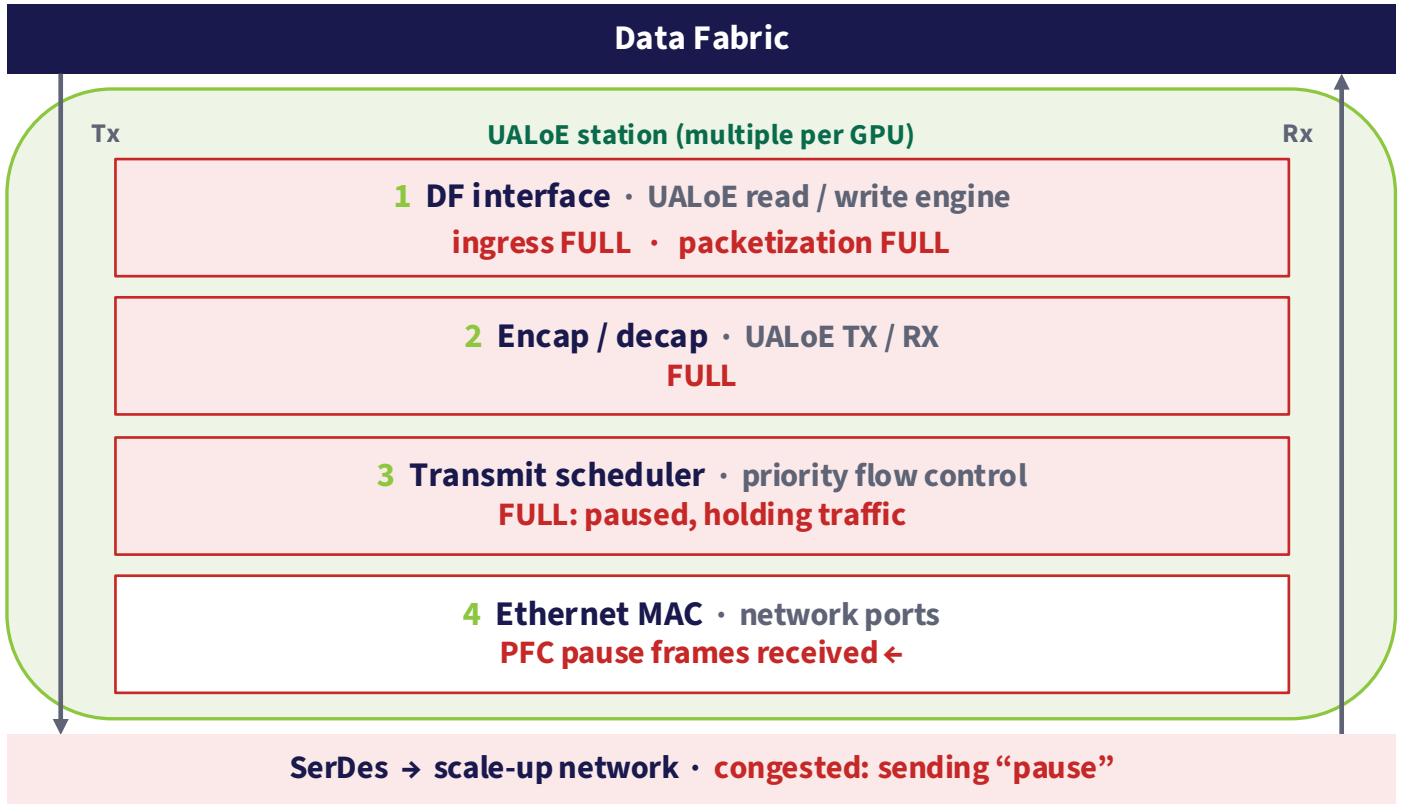
Nothing obliges the next platform, or another vendor, to expose the same set. **That is Call to Action #1.**

Two counters per stage (full and empty), plus credits, stalls and tag lifecycle. The set exists; the standard does not.

Public

Reading a verdict: the scale-up network is pushing back

Backpressure fills every stage upstream of the choke point. Here the full region runs right up to the wire, and the wire is pushing back.



Where does the full region end?

~~X At the DF interface / packetizer~~

~~X Inside the station~~

✓ At the wire: network pushing back

? Source, or victim? One station can't say. Next: the whole platform.

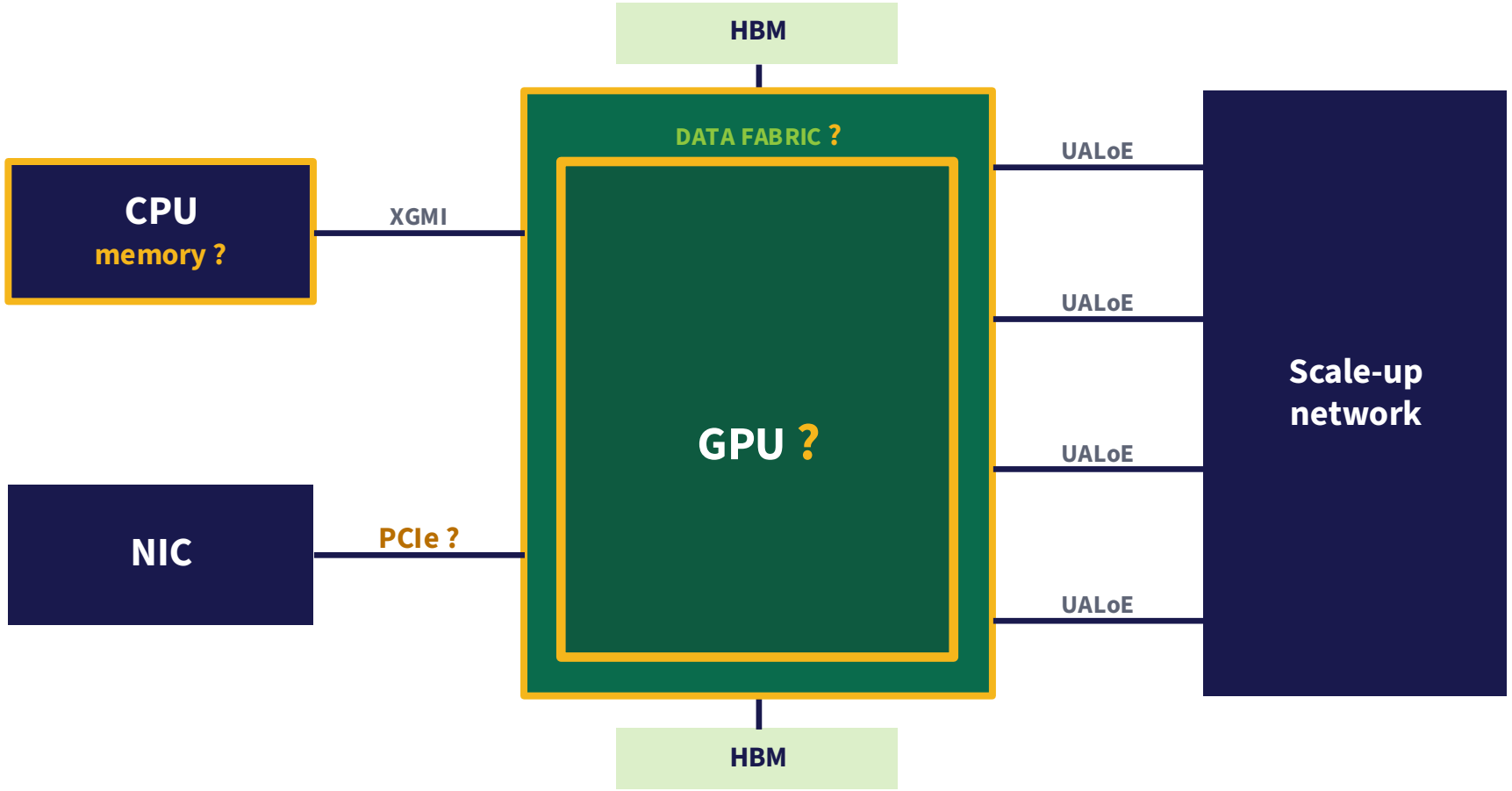
Packetizer-bound: ingress full, packetization empty. Inside the station: stages full, no pause frames.

Where the full region ends, plus one pause counter: a station-level verdict in one read.

Public

PCIe? Memory? GPU? Fabric? The question we can't answer

A job slows down and nothing raises an error. Every link meets at the Data Fabric, and with no common interface telemetry, all four suspects stay open.



Public

Attributing a stall to the interface that caused it

The Data Fabric is shared. When PCIe traffic saturates the path into HBM, UALoE writes to memory back up, and the station sends PFC pauses that make the network look guilty. Follow the backlog to where it ends.

SOURCE where the backlog ends: the point that stops draining, and the traffic filling it

VICTIM full because the shared Data Fabric won't drain; a scale-up victim even sends PFCs

WORKED EXAMPLE: a collective slows down, and the scale-up network sees PFC pauses coming from this GPU

Interface	What its counters show	Read
Scale-up (UALoE)	Rx full · sending PFC pauses · DF credits exhausted on writes to HBM	**victim: looks guilty, isn't**
PCIe → NIC	heavy DMA writes into HBM · own link flowing, no backpressure	**AGGRESSOR: the traffic filling it**
XGMI	writes to HBM backed up · own link clean	victim
Memory (DF → HBM)	write queue pinned full · HBM write bandwidth saturated	**CHOKE POINT: the backlog ends here**

One sweep across four ports replaces a multi-team bisection, and it is falsifiable: a port with no counters can be neither blamed nor cleared.

The scale-up read runs on Helios today. PCIe, XGMI and memory-side counters are defined in silicon or deep-debug tooling, not yet always-on.

Public

Path to an open standard

Every vendor exposes telemetry differently today. We want one schema and one verbosity contract for all of them.

Current state

- Each vendor exposes telemetry differently
- No common register schema across CPU / GPU / NIC
- No standard verbosity contract
- Bring-up visibility gap at every new platform
- Tooling per vendor, per platform

Proposed OCP standard

- Common boundary-telemetry register schema
- **PCIe / XGMI / C2C / CXL:** port latency, credit health, backpressure, bandwidth
- **Verbosity contract:** production vs. debug, in the spec
- **BMC Redfish** as the universal OOB collection plane
- **Day-1 bring-up visibility** on any new platform

> *The vendor with better observability qualifies faster and deploys at scale sooner. Observability is a competitive advantage, not a cost.*

Public

Call to action for the OCP community

Standardise interface-boundary telemetry across all AI silicon

01	**Expose interface telemetry**	Every silicon vendor exposes port latency, credit health, backpressure and bandwidth at every inter-chip interface, via batched, cacheline-aligned APIs
02	**Telemetry impact as a qualification gate**	Platforms that degrade under their own monitoring should not ship: >1% BusBW degradation at production polling intervals = qualification failure
03	**Mandate an OOB collection plane**	BMC Redfish as the universal out-of-band anchor: persistent, immune to in-band congestion, available during boot and runtime
04	**Open reference implementations**	Field-proven tooling open-sourced under OCP: vendor-agnostic input formats, register maps, validation heatmap generators
05	**Establish an OCP working group**	Hyperscalers + silicon vendors + ODMs ratify the boundary-telemetry schema and verbosity contract before the next generation ships

Public

Thank you

Antonis Michaloliakos
Production Systems Engineer, AI Systems, Meta

Nischay Tata
Fellow, Systems Design Engineer, AMD

Meta AI Systems × AMD

Public

COPYRIGHT AND DISCLAIMER

©2026 Advanced Micro Devices, Inc. All rights reserved.

AMD, the AMD Arrow logo, MI300X, MI350X and combinations thereof are trademarks of Advanced Micro Devices, Inc. Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate releases, for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information. However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED 'AS IS." AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Public

